

实验七 查找实验

基础练习一

【问题描述】

题目要求在无序表 A 中用顺序查找找出值为 x 的元素。ex_1 的 sample_data() 保存 10 个样例数据，search() 从表头开始比较，并返回找到的下标。

若查找失败，函数返回 -1；主程序负责输出数组下标、数组值并读取用户输入。

【基本思路】

顺序查找从下标 0 开始逐个比较，找到 key 时立即返回；如果一直比较到表尾还没有相等元素，就返回 -1。无序表不能提前缩小范围，所以最坏情况下要查完整张表。

函数调用关系为：main() -> sample_data() -> search()。

```
search(A, x)
  i <- 0
  while i < len(A) and A[i] != x
    i <- i + 1
  if i >= len(A) return -1
  else return i
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_7/ex_1/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据选取表头、表中和不存在的关键字，分别检查返回下标和失败返回值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	查找 2	返回下标 0，说明表头元素可以被正确找到。
2	查找 15	返回下标 5，和主程序输出 A[5]=15 一致。
3	查找 100	返回 -1，程序输出未找到。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是循环边界，i 增加到 a.len() 时不能再访问 a[i]。当前实现把边界判断写在 while 条件前半部分，避免越界。

2.2 本实验设计的优点及可以改进的方向

可以把输入数据改为文件读取，或统计比较次数，便于观察不同位置元素的查找代价。

基础练习二

【问题描述】

题目要求在长度不超过 1024 的有序表中查找键值 x。ex_2 使用 bin_search() 完成折半查找，返回匹配元素下标，未找到时返回 -1。

样例数据为 3、8、12、15、18、21、25、30、36、42，数据已按升序排列。

【基本思路】

折半查找维护 low 和 high 两个边界，每轮取中间位置 mid。a[mid] 小于 x 时查右半段，a[mid] 大于 x 时查左半段，相等时直接返回 mid。

当 mid 为 0 且还需要继续向左缩小时，程序直接跳出循环，防止 usize 下标减 1 溢出。

```
bin_search(A, x)
  low <- 0, high <- len(A) - 1
  while low <= high
    mid <- (low + high) / 2
    if A[mid] = x return mid
    if A[mid] < x low <- mid + 1
    else high <- mid - 1
  return -1
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_7/ex_2/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据包含表头、表中、表尾和空表等情况，用来检查折半区间收缩是否正确。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
----	------	-------------

1	查找 3	返回下标 0，左端元素查找成功。
2	查找 21	返回下标 5，主程序输出查找成功，位置为 5。
3	空表查找	返回 -1，测试 bin_search_handles_empty_table 通过。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是有序表区间的收缩。Rust 中 `usize` 不能出现负数，所以 `high` 左移时要单独处理 `mid` 等于 0 的情况。

2.2 本实验设计的优点及可以改进的方向

可以继续增加递归版折半查找，与当前循环版比较代码长度和运行过程。

基础练习三

【问题描述】

练习三给出的关键字集合为 {22, 12, 13, 8, 9, 20, 33, 42, 44, 38, 24, 48, 60, 58, 74, 49, 86, 53}。程序需要先建立索引表，再按索引顺序查找。

`ex_3` 把 18 个关键字分成 3 块，每块 6 个元素。块内数据不重新排序，索引表只记录每块起始下标和该块最大关键字。

【基本思路】

`IndexNode` 保存 `start` 和 `max_key`。`build_index()` 逐块求最大关键字；`block_search()` 先用索引表确定候选块，再在这一块里顺序比较。

样例索引表为 (`start=0, max_key=22`)、(`start=6, max_key=48`)、(`start=12, max_key=86`)。

```
block_search(A, index, key)
    在 index 中找到第一个 max_key >= key 的块
    if 没有这样的块 return -1
    i <- block.start
    while i < block.end
        if A[i] = key return i
        i <- i + 1
    return -1
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_7/ex_3/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据检查三块索引的最大关键字、块内查找成功位置，以及超出索引范围时的失败结果。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	建立索引	得到 3 个索引结点，最大关键字分别为 22、48、86。
2	查找 24	先定位到第 2 块，再顺序查找得到下标 10。
3	查找 100	索引表中不存在可容纳该关键字的块，返回 -1。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是块内无序、块间有序。索引表必须保存块起始地址和块内最大关键字，查找时才可以先缩小范围。

2.2 本实验设计的优点及可以改进的方向

目前分块大小固定为 6。若数据量变化较大，可以根据 n 的大小重新选择块长。

进阶练习一

【问题描述】

进阶练习一要求把元素 x 插入有序表，插入后表仍保持升序。ex_4 先用 `bin_insert_pos()` 找位置，再由 `insert_ordered()` 从表尾向后腾出空位。

样例表仍为 3、8、12、15、18、21、25、30、36、42。

【基本思路】

`bin_insert_pos()` 返回第一个不小于 x 的下标。`insert_ordered()` 先在 `Vec` 末尾补一个位置，然后从最后一个元素开始后移，直到空出插入点。

主程序读入一个整数，输出插入位值和插入后的表。

```
insert_ordered(A, x)
  pos <- bin_insert_pos(A, x)
  len <- len(A) + 1
  i <- len - 1
  while i > pos
```

```

    A[i] <- A[i - 1]
    i <- i - 1
    A[pos] <- x
    return pos

```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_7/ex_4/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据覆盖表头插入、表中插入和表尾插入，重点检查插入后 Vec 是否仍为升序。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	插入 20	插入位置为 5，结果为 3 8 12 15 18 20 21 25 30 36 42。
2	插入 1	插入到表头，下标为 0。
3	插入 50	插入到表尾，仍保持升序。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是插入位置和元素搬移方向。若从前向后搬移，会覆盖还没有移动的数据，因此需要从尾部开始后移。

2.2 本实验设计的优点及可以改进的方向

当前实现允许插入重复值，重复值会放在已有相同元素前面；若需要唯一关键字，可在插入前先查找一次。

进阶练习二

【问题描述】

进阶练习二要求用不超过 100 个整数构造二叉排序树。ex_5 定义 BinaryTree 和 TreeNode，并按输入顺序逐个插入样例数据。

样例数据为 45、24、53、12、37、93、28、40、60、70。

【基本思路】

插入时先与当前结点 `key` 比较。若相等则不插入；若较小则进入左子树；若较大则进入右子树。遇到空链接时建立新结点。

程序用 `inorder()` 输出中序序列。二叉排序树建立正确时，中序遍历结果为升序。

```
insert(T, key)
    if T is empty
        建立新结点作为当前位置
    else if key = T.key
        return false
    else if key < T.key
        insert(T.left, key)
    else
        insert(T.right, key)
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 `EXP/EXP_7/ex_5/src/lib.rs`，主程序位于对应 `src/main.rs`，测试代码位于 `tests/test_fn.rs`。

测试数据检查样例树的中序遍历结果、普通查找和重复关键字插入。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	构造样例树	中序序列为 12 24 28 37 40 45 53 60 70 93。
2	查找 37	返回 <code>true</code> ，说明左、右子树递归插入后仍能查找。
3	重复插入 10	第二次插入返回 <code>false</code> ，树中只保留一个 10。

2. 结果及方案分析

2.1 本实验的重点及难点

本题难点在于修改树中某个空链接。`insert_node()` 接收可变链接，遇到 `None` 时建立新结点，遇到已有结点时按 `key` 的大小继续向左或向右走。

2.2 本实验设计的优点及可以改进的方向

可以在后续补充删除结点操作，并对叶子结点、单分支结点和双分支结点分别测试。